

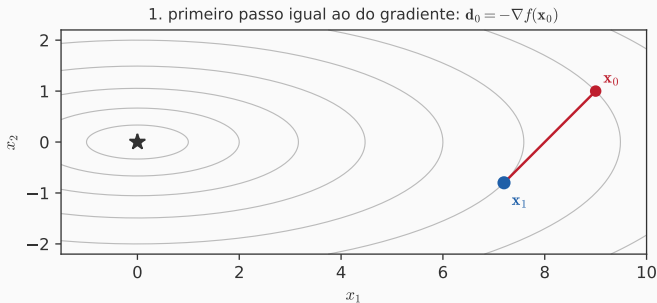
Otimização

3.3.3 Método dos gradientes conjugados (Fletcher–Reeves)

José Firmino Aguilar Madeira

O mesmo começo — onde o gradiente começaria a ziguezaguear

O exemplo do 3.3.1: $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$ a partir de $\mathbf{x}_0 = (9, 1)$, pesquisa em linha exata. O primeiro passo é **igual** ao do gradiente: $\mathbf{d}_0 = -\nabla f(\mathbf{x}_0)$, e chega-se a $\mathbf{x}_1 = (7, 2; -0,8)$.

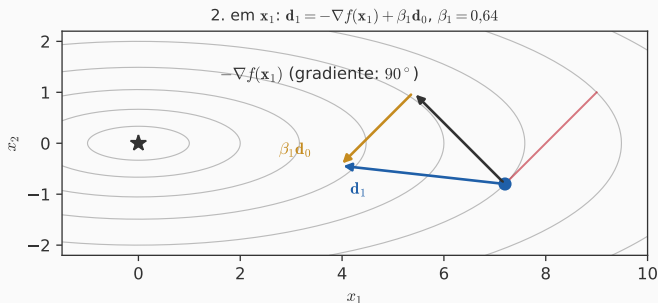


Em $\mathbf{x}_1$, o gradiente viraria 90° e começaria o ziguezague (3.3.1). A pergunta: há uma direção melhor, usando só gradientes?

Lembrar a direção anterior

Em vez de seguir só $-\nabla f(\mathbf{x}_1)$, junta-se uma parte da direção anterior: $\mathbf{d}_1 = -\nabla f(\mathbf{x}_1) + \beta_1 \mathbf{d}_0$, com

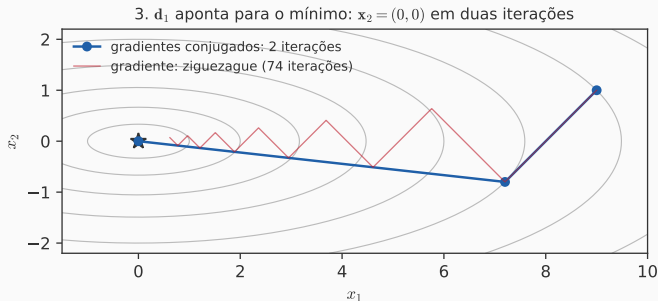
$$\beta_1 = \frac{\|\nabla f(\mathbf{x}_1)\|^2}{\|\nabla f(\mathbf{x}_0)\|^2} = \frac{103,68}{162} = 0,64 \text{ (Fletcher-Reeves).}$$



Custo extra: nenhum gradiente a mais — só guardar $\mathbf{d}_0$ e $\|\nabla f(\mathbf{x}_0)\|^2$, e dois produtos internos.

Duas iterações em vez de 74

Com $\mathbf{d}_1 = (-12,96; 1,44)$ e pesquisa em linha exata ($\alpha_1 = 5/9$): $\mathbf{x}_2 = (0, 0)$, o mínimo, em duas iterações ($n = 2$).

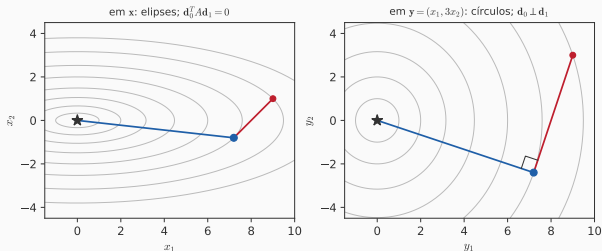


O gradiente precisou de 74 iterações para $\|\nabla f\| < 10^{-6}$, com a mesma informação por iteração.

O que quer dizer “conjugada”?

$\mathbf{d}_0$ e $\mathbf{d}_1$ não são perpendiculares, mas são **A-conjugadas**, com $A = \nabla^2 f = \text{diag}(1, 9)$:

$\mathbf{d}_0^T A \mathbf{d}_1 = (-9, -9) (-12, 96; 12, 96)^T = 116,64 - 116,64 = 0$. Mudando a escala, $y_1 = x_1$, $y_2 = 3x_2$, as elipses tornam-se círculos — e as direções conjugadas tornam-se **perpendiculares**.



Com círculos, dois passos perpendiculares com pesquisa exata chegam ao centro. “Conjugadas” = perpendiculares na geometria de A : minimizar ao longo de $\mathbf{d}_1$ não estraga o que se ganhou ao longo de $\mathbf{d}_0$.

Agora formalmente

Quadrática $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$ com $A = A^T \succ 0$ (SPD). Direções **A-conjugadas**:

$$\mathbf{d}_i^T A \mathbf{d}_j = 0 \quad (i \neq j) \quad (\text{no exemplo: } \mathbf{d}_0^T A \mathbf{d}_1 = 0).$$

Resultado

Quadrática SPD + aritmética exata + pesquisas em linha exatas ao longo de direções A-conjugadas
 $\Rightarrow$ o mínimo em no máximo n iterações.

Fletcher e Reeves (1964) geram essas direções *só com gradientes* ($\mathbf{g}_k = \nabla f(\mathbf{x}_k)$):

$$\mathbf{d}_0 = -\mathbf{g}_0, \quad \mathbf{d}_k = -\mathbf{g}_k + \beta_k \mathbf{d}_{k-1}, \quad \beta_k^{FR} = \frac{\mathbf{g}_k^T \mathbf{g}_k}{\mathbf{g}_{k-1}^T \mathbf{g}_{k-1}} = \frac{\|\mathbf{g}_k\|^2}{\|\mathbf{g}_{k-1}\|^2}.$$

Gradiente (3.3.1): gradientes consecutivos ortogonais, $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$

Gradientes conjugados: direções A-conjugadas, $\mathbf{d}_i^T A \mathbf{d}_j = 0$: ortogonais “na geometria de A”

O algoritmo (Fletcher–Reeves)

Fletcher–Reeves: $\mathbf{x}_0, \varepsilon_g, k_{\max}$

```
1:  $\mathbf{g}_0 \leftarrow \nabla f(\mathbf{x}_0)$ ;  $\mathbf{d}_0 \leftarrow -\mathbf{g}_0$ ;  $k \leftarrow 0$ 
2: loop
3:    $\alpha_k \leftarrow \arg \min_{\alpha > 0} f(\mathbf{x}_k + \alpha \mathbf{d}_k)$   $\triangleright$  pesquisa em linha
4:    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{d}_k$ ;  $\mathbf{g}_{k+1} \leftarrow \nabla f(\mathbf{x}_{k+1})$ 
5:   if  $\|\mathbf{g}_{k+1}\| \leq \varepsilon_g$  or  $k \geq k_{\max}$  then return  $\mathbf{x}_{k+1}$ 
6:   end if
7:    $\beta_{k+1} \leftarrow \|\mathbf{g}_{k+1}\|^2 / \|\mathbf{g}_k\|^2$ 
8:    $\mathbf{d}_{k+1} \leftarrow -\mathbf{g}_{k+1} + \beta_{k+1} \mathbf{d}_k$ ;  $k \leftarrow k + 1$ 
9: end loop
```

$\arg \min_{\alpha > 0} \varphi(\alpha)$: o valor de α onde φ atinge o mínimo — o minimizante, não o valor mínimo (2.1).

gradiente atual + **memória** da
direção anterior

- Custo por iteração: o do método do gradiente ($n_g = 1$ + as avaliações de f da pesquisa em linha) mais dois produtos internos.
- Memória: $\mathbf{x}_k, \mathbf{g}_k, \mathbf{d}_k \sim O(n)$, sem Hessiana.
- Na quadrática SPD, com pesquisa exata e aritmética exata: no máximo n iterações. E fora das quadráticas?

Fora das quadráticas: porquê e quando reiniciar

Reiniciar = fazer $\beta_{k+1} = 0$, isto é, voltar a $\mathbf{d}_{k+1} = -\mathbf{g}_{k+1}$. **Porquê:** a conjugação foi construída para uma quadrática com pesquisa exata; numa f não quadrática, ou com pesquisa inexata, perde-se aos poucos e a “memória” passa a atrapalhar.

Quando reiniciar	Teste	Porquê
1. A cada n iterações	$(k+1) \bmod n = 0$	numa quadrática bastam n direções
2. $\mathbf{d}_{k+1}$ deixa de ser de descida	$\mathbf{g}_{k+1}^T \mathbf{d}_{k+1} \geq 0$	a pesquisa em linha não baixaria f
3. Gradientes consecutivos longe de ortogonais	ver nota*	a conjugação perdeu-se

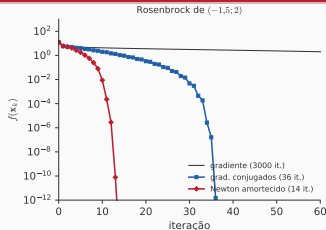
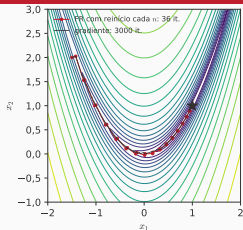
* Critério de Powell (1977): reiniciar se $|\mathbf{g}_{k+1}^T \mathbf{g}_k| \geq \nu \|\mathbf{g}_{k+1}\|^2$, com $\nu \approx 0,1-0,2$ (com pesquisa exata numa quadrática, $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$).

Efeito no Rosenbrock (página seguinte): FR **sem** reinício, 102 iterações; **com** reinício a cada $n = 2$, 36. O MATLAB destes apontamentos usa os testes 1 e 2.

Outra escolha comum de β : Polak–Ribière⁺

$\beta_k^{PR+} = \max\left(0, \mathbf{g}_k^T (\mathbf{g}_k - \mathbf{g}_{k-1}) / \|\mathbf{g}_{k-1}\|^2\right)$ — fora das quadráticas, reinicia sozinha ($\beta \geq 0$): quando a fração é negativa, $\beta = 0$.

Rosenbrock: gradiente vs. gradientes conjugados vs. Newton



- Rosenbrock não é quadrático, mas perto do mínimo parece: FR com reinício a cada $n = 2$ iterações converge em 36 iterações ($\|\nabla f\| < 10^{-4}$); sem reinício, 102; o gradiente não converge em 3000.
- Newton amortecido: 14 iterações — mas cada uma custa uma Hessiana. Contado até $f < 10^{-4}$, com a mesma pesquisa em linha de elevada precisão (Brent, tol. 10^{-10}): Newton $n_g = n_H = 12$, $n_f = 269$ na pesquisa em linha (281 com f nos iterandos); FR $n_g = 34$, $n_f = 684$ (718). Em $n = 2$ a pesquisa em linha é o maior custo; à medida que n cresce, formar, armazenar e fatorizar a Hessiana torna os métodos que só usam gradientes progressivamente mais atraentes.

número de iterações $\neq$ custo

O núcleo, em MATLAB:

```
g = grad(x); d = -g;           % 1.a direcao
% ... em cada iteracao:
alpha = ...;                   % pesq. em linha
x = x + alpha*d;
gn = grad(x);
beta = (gn'*gn)/(g'*g);        % Fletcher-Reeves
d = -gn + beta*d;
if mod(k+1,n)==0 || gn'*d >= 0
    d = -gn;                   % reinicio
end
g = gn;
```

A função completa (ConjGrad, com info.nfev, info.ngcv) está no código complementar da UC. Na quadrática: 2 iterações, $n_g = 3$.

A pesquisa em linha não é um detalhe

Na quadrática, a conjugação exata pressupõe pesquisa em linha exata ($\mathbf{g}_{k+1}^T \mathbf{d}_k = 0$). Em problemas não lineares usam-se normalmente pesquisas *inexatas* suficientemente rigorosas (condições de Wolfe), juntamente com reinícios.

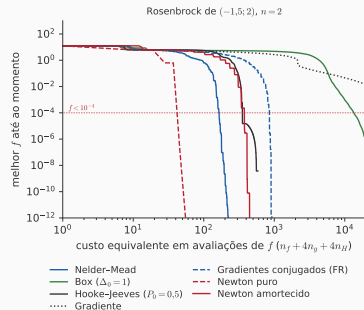
Com o fminbnd de tolerância por defeito, FR no Rosenbrock pode não convergir: **dos três métodos estudados aqui, é particularmente sensível à qualidade da pesquisa em linha.**

Os métodos estudados no mesmo problema, com o custo em avaliações

Rosenbrock de $(-1,5; 2)$; paragem: primeira avaliação (ou iterando) com $f < 10^{-4}$ (f nos iterandos conta em n_f); em 3.3, pesquisa em linha precisa (Brent, tol. 10^{-10}). A grelha (3.2.2), para explorar e inicializar, fica fora.

Método	Derivadas (ordem)	n_f	n_g	n_H	equiv. em f
Aleatória localizada [†]	0	975 [867; 1052]	—	—	975
Nelder–Mead	0	166	—	—	166
Box ($\Delta_0 = 1$)	0	13 445	—	—	13 445
Hooke–Jeeves ($P_0 = 0,5$)	0	353	—	—	353
Gradiente [‡]	1	60 900	3000	—	72 900
Grad. conj. (FR, reinício n)	1	718	34	—	854
Newton puro	2	6	5	5	46
Newton amortecido	2	281	12	12	377

[†] estocástico: mediana [quartis] de 30 corridas. [‡] não atinge $f < 10^{-4}$ em 3000 iterações ($f = 2,4 \cdot 10^{-4}$). Equiv. em f (derivadas centrais, $n = 2$): $n_f + 4n_g + 4n_H$ (a Hessiana, $2n^2 + 1 = 9$ pontos, reaproveita os 4 do gradiente e $f(x_k)$).



Neste problema, com estes parâmetros: Newton puro tem o menor custo quando converge para a solução pretendida; Nelder–Mead bate os métodos de ordem 1 porque em $n = 2$ a pesquisa em linha precisa custa ≈ 20 avaliações por iteração. Com n grande ($n = 100$: gradiente numérico 200, Hessiana 20 001) os métodos só com gradientes ganham terreno — não há vencedor em abstrato.

Quadro de consulta — 3.3

Que informação temos sobre f ? Quanto custa obtê-la? O que conseguimos garantir com ela?

Método	Informação	Custo por iteração	Garantia	Principal limitação
Gradiente (3.3.1)	∇f	$n_g = 1$; n_f = custo da pesq. em linha; álgebra $O(n)$	estacionariedade de qualquer $\mathbf{x}_0$ (nível limitado, gradiente Lipschitz, pesquisa adequada); linear, cota $((\kappa - 1)/(\kappa + 1))^2$ (quadrática SPD, pesquisa exata)	zigzague se κ grande
Newton (3.3.2)	$\nabla f, \nabla^2 f$	$n_g = n_H = 1 +$ sistema $n \times n$ ($O(n^3)$; memória $O(n^2)$)	local quadrática se $H(\mathbf{x}^*) \succ 0$ (sob regularidade; amortecido: $\alpha_k \rightarrow 1$ com Armijo/Wolfe a partir de $\alpha = 1$); 1 it. em quadráticas SPD	Hessiana cara; sem amortecimento pode divergir ou ir para máximos/selas
Grad. conj. (3.3.3)	∇f + direção anterior	$n_g = 1$; n_f = custo da pesq. em linha; memória $O(n)$	$\leq n$ it. em quadráticas SPD (pesquisa exata, aritmética exata); em geral, $\liminf \ \nabla f(\mathbf{x}_k)\ = 0$ com as hipóteses do gradiente e pesquisa exata ou de Wolfe forte (Al-Baali, 1985)	sensível à pesquisa em linha

Rosenbrock, pesquisa em linha precisa, paragem $\|\nabla f\| < 10^{-4}$: gradiente 3000 it. sem a atingir ($n_g = 3000$); Newton amortecido 14 it. ($n_g = 15$, $n_H = 14$); FR com reinício 36 it. ($n_g = 37$). Gradiente se n é enorme e a precisão modesta; Newton se a Hessiana é barata; gradientes conjugados e quasi-Newton são compromissos entre custo por iteração e uso de curvatura/memória.

O QUE DEVE FICAR — 3.3.3

1. Gradientes conjugados = gradiente atual + **memória** da direção anterior:
 $\mathbf{d}_k = -\mathbf{g}_k + \beta_k \mathbf{d}_{k-1}$ (Fletcher–Reeves).
2. Direções **A-conjugadas**: numa quadrática SPD, com pesquisa exata e aritmética exata, o mínimo em **no máximo n iterações**.
3. Custo por iteração quase igual ao do gradiente, memória $O(n)$, sem Hessiana: um compromisso atraente para n grande.
4. Fora das quadráticas: **reinícios** e pesquisa em linha rigorosa — dos três métodos, é o mais sensível a ela.
5. Número de iterações $\neq$ custo: compara-se em avaliações, neste problema, com estes parâmetros.

O QUE DEVE FICAR — capítulo 3

1. Em várias dimensões, nos métodos baseados em direções, cada iteração escolhe uma **direção** e um **passo**; os métodos diferem na informação que usam.
2. **Sem derivadas** (3.2): só valores de f , para explorar ou refinar localmente. O NM não tem garantia geral; o HJ está na origem da pesquisa em padrão, cujas versões formalizadas (GPS) têm estacionariedade no limite sob hipóteses adequadas.
3. **Com derivadas** (3.3): o gradiente é barato mas lento se mal condicionado; Newton é rápido perto do mínimo, mas caro e com salvaguardas; os gradientes conjugados ficam no meio.
4. O custo conta-se em **avaliações**, não em iterações; não há vencedor em abstrato.
5. Os métodos locais garantem, no melhor caso, **estacionariedade** — não o mínimo global.

A seguir (cap. 4): **e se nem todas as direções forem permitidas?**

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Fletcher, R., Reeves, C. M., *Function minimization by conjugate gradients*, The Computer Journal 7(2):149–154, 1964.
- Powell, M. J. D., *Restart procedures for the conjugate gradient method*, Mathematical Programming 12(1):241–254, 1977.
- Al-Baali, M., *Descent property and global convergence of the Fletcher–Reeves method with inexact line search*, IMA Journal of Numerical Analysis 5(1):121–124, 1985.
- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2.^a ed., Springer, 2006 — cap. 5.

Recursos

- Código complementar: ConjGrad (info.nfev, info.ngev). MATLAB: fminbnd.
- Caderno cap3_comparacao.ipynb: reproduz a tabela comparativa do fim do capítulo.